

Modus Ponens Generalizzato (MPG)

$$\frac{p'_1, p'_2, \dots, p'_n, \quad p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q}{q\theta}$$

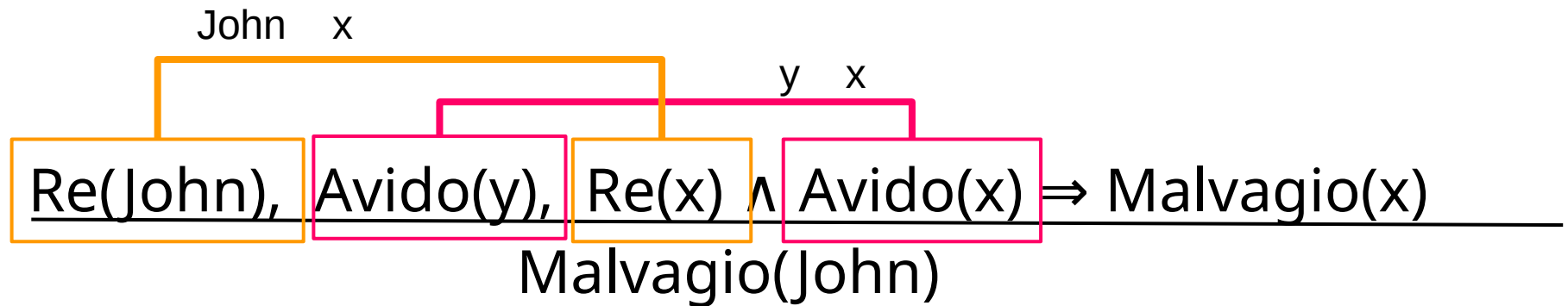
Dove $p'_i\theta = p_i\theta$ per ogni $i \in [1, n]$

La regola ha come premesse n formule atomiche e una singola implicazione. La conclusione è il risultato dell'applicazione della sostituzione θ alla formula q , conseguenza dell'implicazione

NB: $q\theta$ è un modo sintetico per scrivere $\text{SUBST}(\theta, q)$

$\text{Re}(\text{John}), \text{Avido}(y), \text{Re}(x) \wedge \text{Avido}(x) \Rightarrow \text{Malvagio}(x)$
 $\text{Malvagio}(\text{John})$

Esempio



Questa conclusione si appoggia alla sostituzione $\{x/\text{John}, y/\text{John}\}$

Le clausole focalizzano la ricerca della sostituzione e permettono di ragionare direttamente in FOL

Nella proposizionalizzazione, di contro, si costruiscono sostituzioni usando in modo esaustivo l'intero vocabolario di costanti

Sulla generalità del MPG: Lifting

- Il modus ponens non pone restrizioni sulla forma della formula antecedente dell'implicazione
- Il modus ponens generalizzato richiede invece che sia una congiunzione
- L'appellativo "generalizzato" deriva dall'aver "sollevato" la regola dalla logica proposizionale a quella del prim'ordine. Questo processo si chiama **lifting**
- Inoltre consente di avere considerare un numero qualsiasi di formule da cui trarre la conclusione (invece di due solamente)

Unificazione

- **Unificazione**: algoritmo chiave di tutte le tecniche di inferenza sul prim'ordine:
 - Date due formule $F1$ e $F2$
 - $UNIFY(F1, F2) = \theta$ tale che $F1 \theta = F2 \theta$
- Il risultato è una sostituzione che, applicata a entrambe le formule, le rende identiche
- Tale sostituzione, se esiste, è detta **UNIFICATORE**
- Se vi sono più unificatori si calcola e si usa il **Most General Unifier** (**MGU, unificatore più generale**)

Esempi

- UNIFY(knows(John, x), knows(John, Richard))
 - {x/Richard}
- UNIFY(knows(John, x), knows(y, Richard))
 - {x/Richard, y/John}
- UNIFY(knows(John, x), knows(y, MotherOf(y)))
 - { x/MotherOf(John), y/John }
- UNIFY(knows(John, x), knows(x, Richard))
 - **Fallisce** perché x non può assumere due valori contemporaneamente
 - **Standardizzazione separata (standardizing apart)**: rinoma delle variabili di una formula per evitare collisioni con un'altra formula.

Abbiamo visto

- Per rispondere alle query occorre fare **inferenza**
- L'inferenza del prim'ordine è diversa da quella proposizionale: abbiamo **quantificatori e variabili**
- È possibile ridurre una KB del prim'ordine in una proposizionale e poi applicare algoritmi di inferenza proposizionali:
 - Se la KB **non contiene quantificatori esistenziali**, quella trasformata sarà equivalente a quella FOL
 - Se la **KB contiene quantificatori esistenziali**, non sarà equivalente ma sarà inferenzialmente equivalente (se derivo una formula dalla seconda tale formula sarà derivabile anche da quella originaria)
- **Rispondere alle query richiede di identificare opportune sostituzioni**
- Tali sostituzioni si trovano più efficientemente se se si usano regole di inferenza di cui è stato fatto il **lifting al prim'ordine**
- È necessario tradurre la KB in **clausole di Horn del prim'ordine**

Clausole di Horn del prim'ordine

- La definizione è analoga a quelle delle clausole di Horn (e clausole definite) proposizionali, sono **disgiunzioni di letterali di cui al più uno è positivo**:
 - **Atomiche**:
 - Avido(x) la variabile è intesa come universalmente quantificata
 - Avido(John)
 - **Implicazione il cui antecedente è costituito da letterali positivi**
 $Re(x) \wedge Avido(x) \Rightarrow Malvagio(x)$
- Non tutte le KB possono essere tradotte in clausole di Horn ma molte possono. A queste è possibile applicare il forward chaining (e il modus ponens generalizzato) per fare inferenze

Una KB di esempio

Sapendo: “La legge dice che è un reato per un negozio vendere alcolici a un minorenne. Marco, minorenne, possiede della birra. Tale birra gli è stata venduta del minimarket Sotto Casa”

Obiettivo: “Dimostrare la reità di Sotto Casa”

Useremo questo esempio per imparare a scrivere una KB in clausole di Horn del prim'ordine e come funzionano le inferenze nel prim'ordine

Una KB di esempio: simboli

Costanti: Marco, SottoCasa

Predicati: Vende, Negozio, Supermarket, Birra, Alcolico, Minorenne, Possiede, Reo

Funzioni: nessuna

Interpretazione:

- Marco: il protagonista della storia
- Sotto Casa: minimarket presso il quale Marco fa acquisti abitualmente
- Vende: relazione ternaria che lega chi ha venduto qualcosa a chi
- Negozio: relazione unaria che indica la proprietà di essere un negozio
- ...

Una KB di esempio

La legge dice che è un reato per un negozio vendere alcolici a un minorenne.

$\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$

Una KB di esempio

La legge dice che è un reato per un negozio vendere alcolici a un minorenne.

$\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$

Marco possiede della birra.

$\exists x \text{Possiede}(\text{Marco}, x) \wedge \text{Birra}(x)$

Applicando la regola di istanziamento dell'esistenziale (EI) questa formula si traduce nelle due formule atomiche:

$\text{Possiede}(\text{Marco}, B), \text{Birra}(B)$

B è una nuova costante, non utilizzata altrove. È la birra comperata da Marco. Le diamo il nome "B". Le costanti come B sono dette costanti di Skolem.

Una KB di esempio

La legge dice che è un reato per un negozio vendere alcolici a un minorenne.

$\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$

Marco possiede della birra.

$\text{Possiede}(\text{Marco}, B), \text{Birra}(B)$

Tale birra gli è stata venduta del minimarket Sotto Casa. Esprimiamo che se Marco possiede della birra l'ha comperata al minimarket

$\text{Possiede}(\text{Marco}, x) \wedge \text{Birra}(x) \Rightarrow \text{Vende}(\text{SottoCasa}, x, \text{Marco})$

La birra è un alcolico

$\text{Birra}(x) \Rightarrow \text{Alcolico}(x)$

Una KB di esempio

$\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$

$\text{Possiede}(\text{Marco}, B), \text{Birra}(B)$

$\text{Possiede}(\text{Marco}, x) \wedge \text{Birra}(x) \Rightarrow \text{Vende}(\text{SottoCasa}, x, \text{Marco})$

$\text{Birra}(x) \Rightarrow \text{Alcolico}(x)$

Poi ancora

$\text{Minimarket}(\text{SottoCasa})$

$\text{Minimarket}(x) \Rightarrow \text{Negozio}(x)$

$\text{Minorenne}(\text{Marco})$

La KB in clausole di Horn

C1) $\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$

C2) $\text{Possiede}(\text{Marco}, B)$

C3) $\text{Birra}(B)$

C4) $\text{Possiede}(\text{Marco}, x) \wedge \text{Birra}(x) \Rightarrow \text{Vende}(\text{SottoCasa}, x, \text{Marco})$

C5) $\text{Birra}(x) \Rightarrow \text{Alcolico}(x)$

C6) $\text{Minimarket}(\text{SottoCasa})$

C7) $\text{Minimarket}(x) \Rightarrow \text{Negozio}(x)$

C8) $\text{Minorenne}(\text{Marco})$

NOTA: questa KB è costituita da **clausole di Horn del prim'ordine**. In particolare si tratta di clausole che non fanno uso di funzioni. Questa particolare classe di KB è detta **DATALOG**

La KB in clausole di Horn

- È l'unica modellazione possibile?
- **Probabilmente no**
- In generale dato un problema di rappresentazione esisteranno diverse alternative
- George Box (statistico):

“Now it would be very remarkable if any system existing in the real world could be exactly represented by any simple model. However, cunningly chosen parsimonious models often do provide remarkably useful approximations. For example, the law $PV = RT$ relating pressure P , volume V and temperature T of an "ideal" gas via a constant R is not exactly true for any real gas, but it frequently provides a useful approximation and furthermore its structure is informative since it springs from a physical view of the behavior of gas molecules. For such a model there is no need to ask the question "Is the model true?". If "truth" is to be the "whole truth" the answer must be "No". The only question of interest is "Is the model illuminating and useful?"

Riassunto talvolta in: **tutti i modelli sono “sbagliati”, alcuni sono utili**

Forward Chaining

- L'algoritmo è simile a quello usato nel caso proposizionale
- Le variabili richiedono qualche cautela: un fatto è la **rinomina** di un altro se sono identici tranne nei nomi delle variabili



Esempio di rinomina:

Fratello(John, x) e Fratello(John, y)

- **Nel calcolo proposizionale un fatto inferito viene aggiunto alla KB solo se non vi compare già**
- **In FOL bisogna fare un test più complesso: un fatto è aggiunto alla KB solo se non è una rinomina di uno già presente**

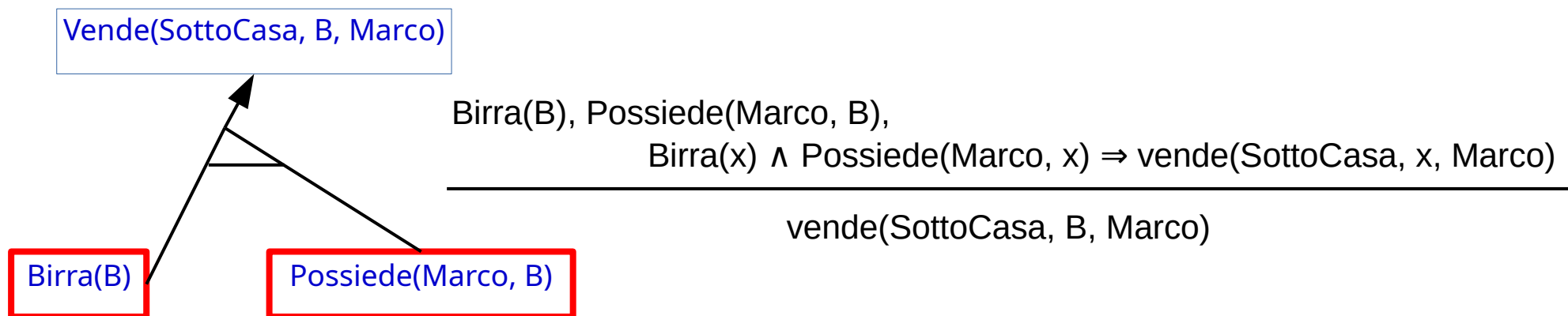
Forward chaining

C2) Possiede(Marco, B)

C3) Birra(B)

C4) **Possiede(Marco, x) $\wedge$ Birra(x) $\Rightarrow$ Vende(SottoCasa, x, Marco)**

Si applica il modus ponens generalizzato



Parto da fatti che rendono vero l'antecedente (la premessa) di una implicazione

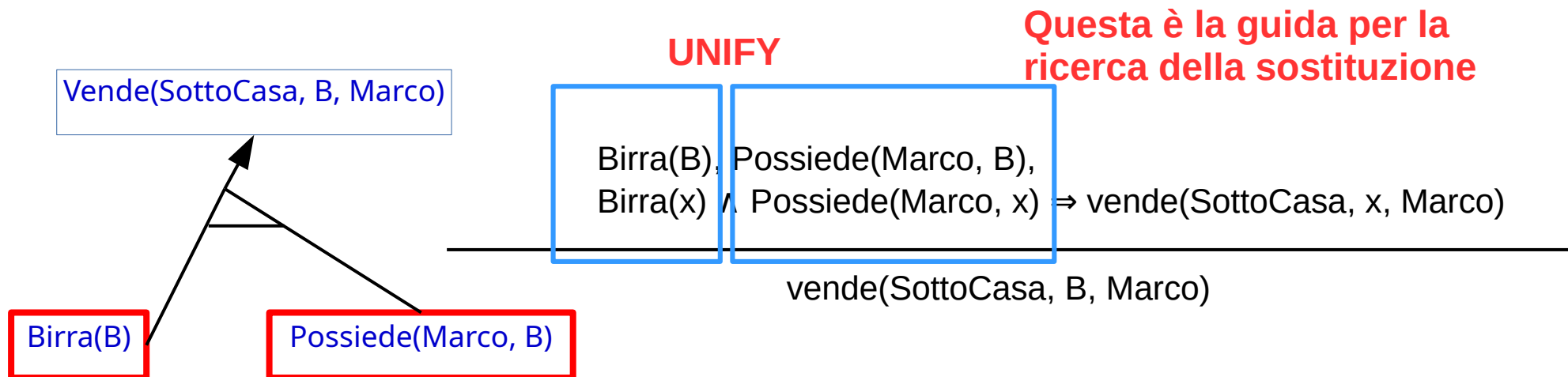
Forward chaining

C2) Possiede(Marco, B)

C3) Birra(B)

C4) **Possiede(Marco, x) $\wedge$ Birra(x) $\Rightarrow$ Vende(SottoCasa, x, Marco)**

Si applica il modus ponens generalizzato



Parto da fatti che rendono vero l'antecedente (la premessa) di una implicazione e costruisco un grafo AND-OR

Forward chaining

C2) Possiede(Marco, B)

C3) Birra(B)

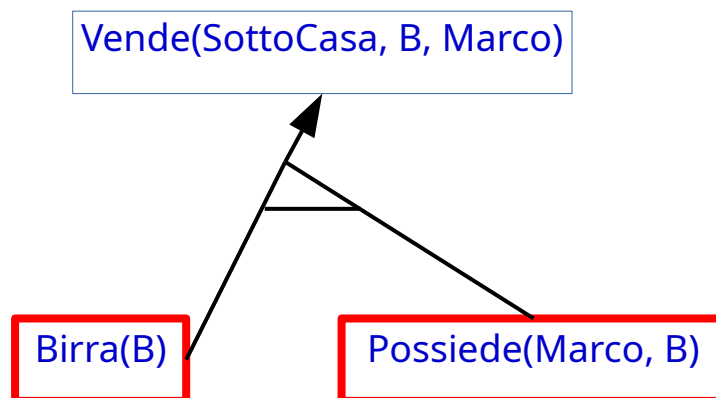
C4) **Possiede(Marco, x) $\wedge$ Birra(x) $\Rightarrow$ Vende(SottoCasa, x, Marco)**

Questa inferenza è possibile legando la variabile x a B.

L'inferenza procede creando delle **sostituzioni** per unificazione delle formule:

$$\theta = \{ x/B \}$$

$$\begin{aligned} \text{Birra}(B)/B &= \text{Birra}(x)/B \\ \text{Possiede}(\text{Marco}, x)/B &= \\ &\text{Possiede}(\text{Marco}, B)/B \end{aligned}$$



Parto da fatti che rendono vero l'antecedente (la premessa) di una implicazione e costruisco un grafo AND-OR

Forward Chaining

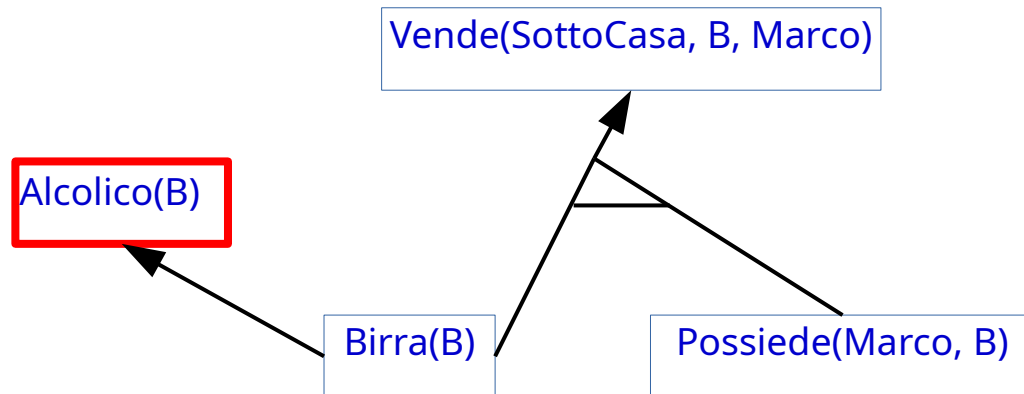
C2) Possiede(Marco, B)

C3) Birra(B)

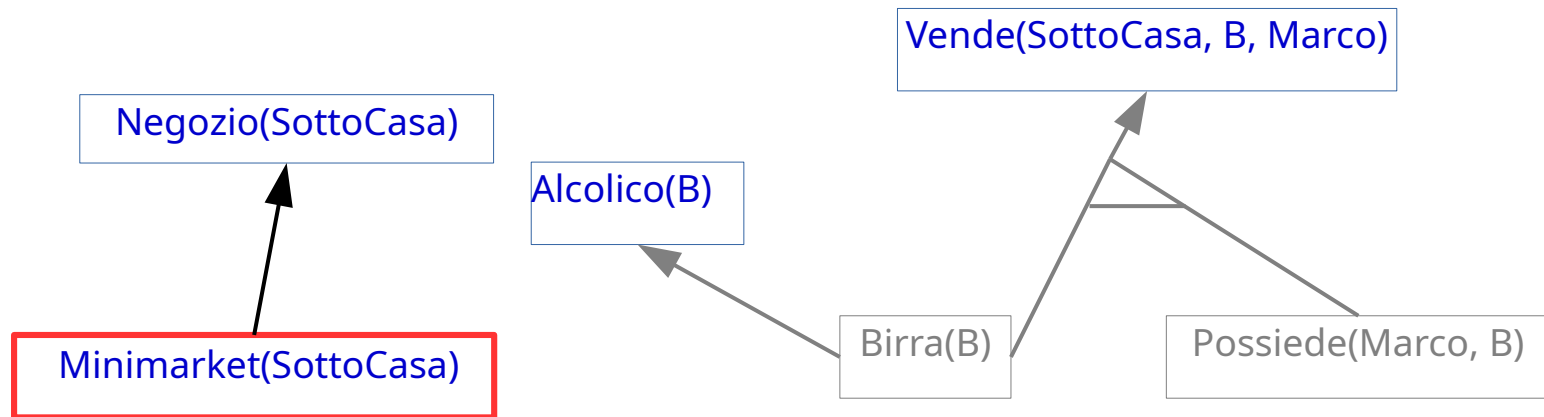
C4) Possiede(Marco, x) $\wedge$ Birra(x) $\Rightarrow$ Vende(SottoCasa, x, Marco)

C5) Birra(x) $\Rightarrow$ Alcolico(x)

Non si propagano solo i valori di verità. Si propagano anche le sostituzioni, che fanno da collante fra le diverse applicazioni del MPG

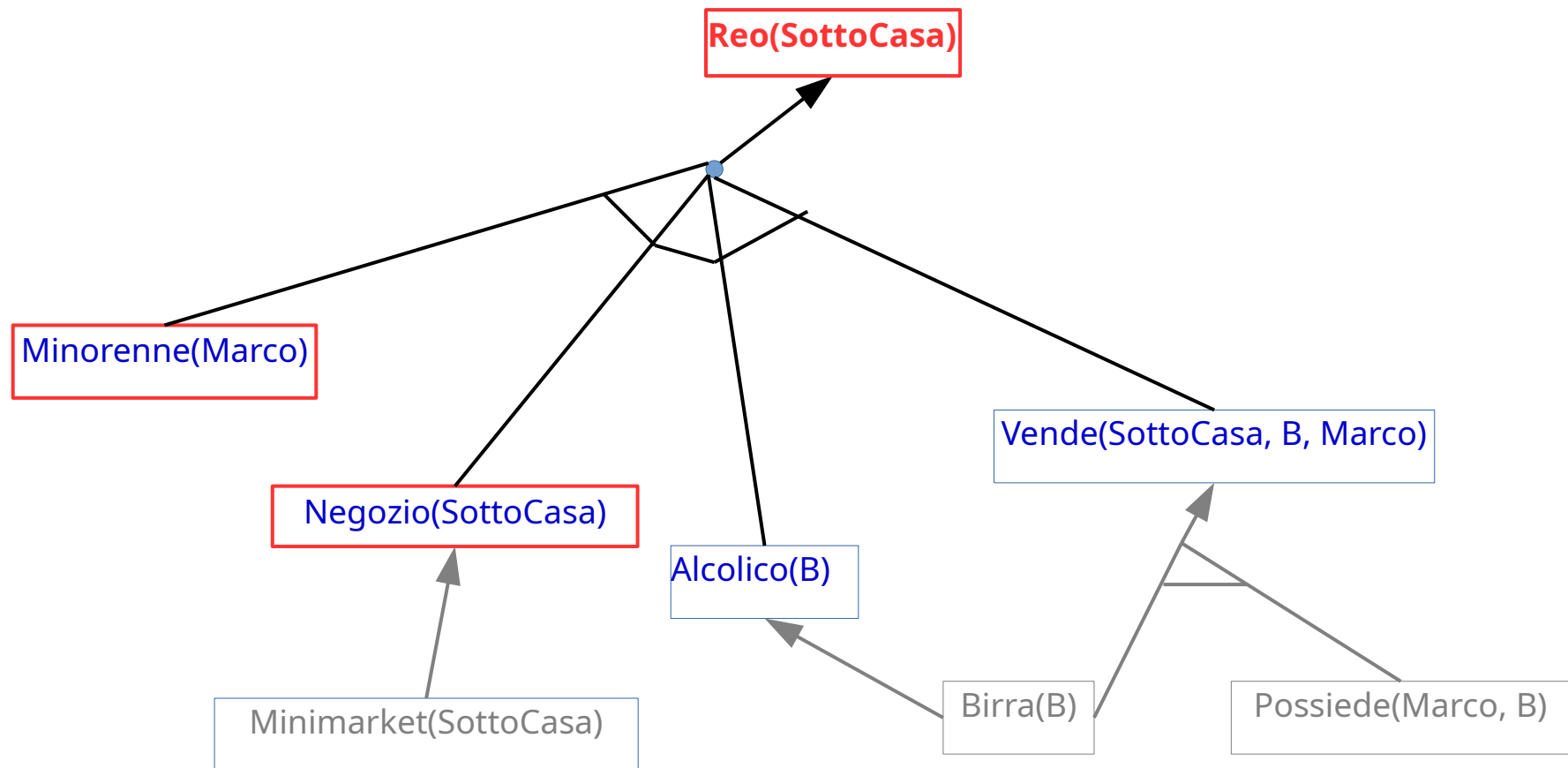


Uso del forward chaining



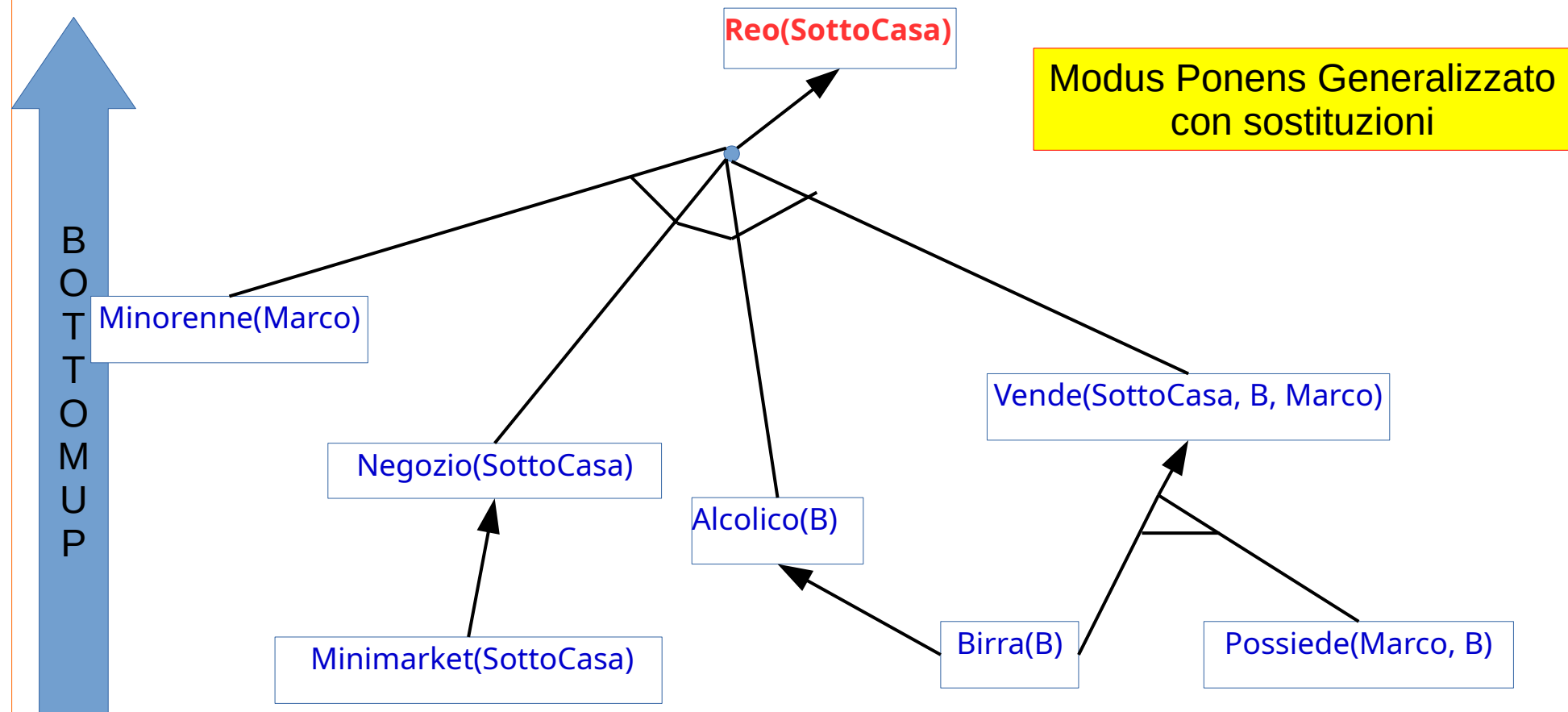
Uso del forward chaining

Inferisco che Sotto Casa è reo della vendita



Uso del forward chaining

Grafo AND-OR completo



Proprietà del forward chaining

- **Correttezza**
 - l'algoritmo è **corretto** perché si basa sulla regola di inferenza corretta GMP
- **Completezza**
 - È **completo** per KB costituite da **clausole di Horn**?
 - Se la KB è **DATALOG**, è **completo e termina**
 - Se la KB **prevede funzioni** – teorema di Herbrand – l'algoritmo **può non terminare** se la risposta non è implicata

Backward Chaining (BC)

- Come nel caso proposizionale il **ragionamento è guidato dall'obiettivo**
- L'obiettivo viene inserito in uno **stack**
- Poi iterativamente si estrae un obiettivo dallo stack e si cercano **le clausole la cui testa è unificabile** con l'obiettivo:
 - Quando un obiettivo unifica con un fatto (clausola con corpo vuoto), viene semplicemente rimosso (è risolto)
 - Altrimenti per ogni clausola che soddisfa quanto sopra si avvia un procedimento ricorsivo che inserisce nella pila le premesse della clausola, in cui le variabili saranno state debitamente sostituite
- Quando la pila è vuota si termina con successo
- Altrimenti se non è possibile applicare altre inferenze si termina con fallimento

Esempio

Stack: Reo(SottoCasa)

Unifica con la testa della clausola tramite $\theta_1 = \{x/\text{SottoCasa}\}$:

C1) $\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$

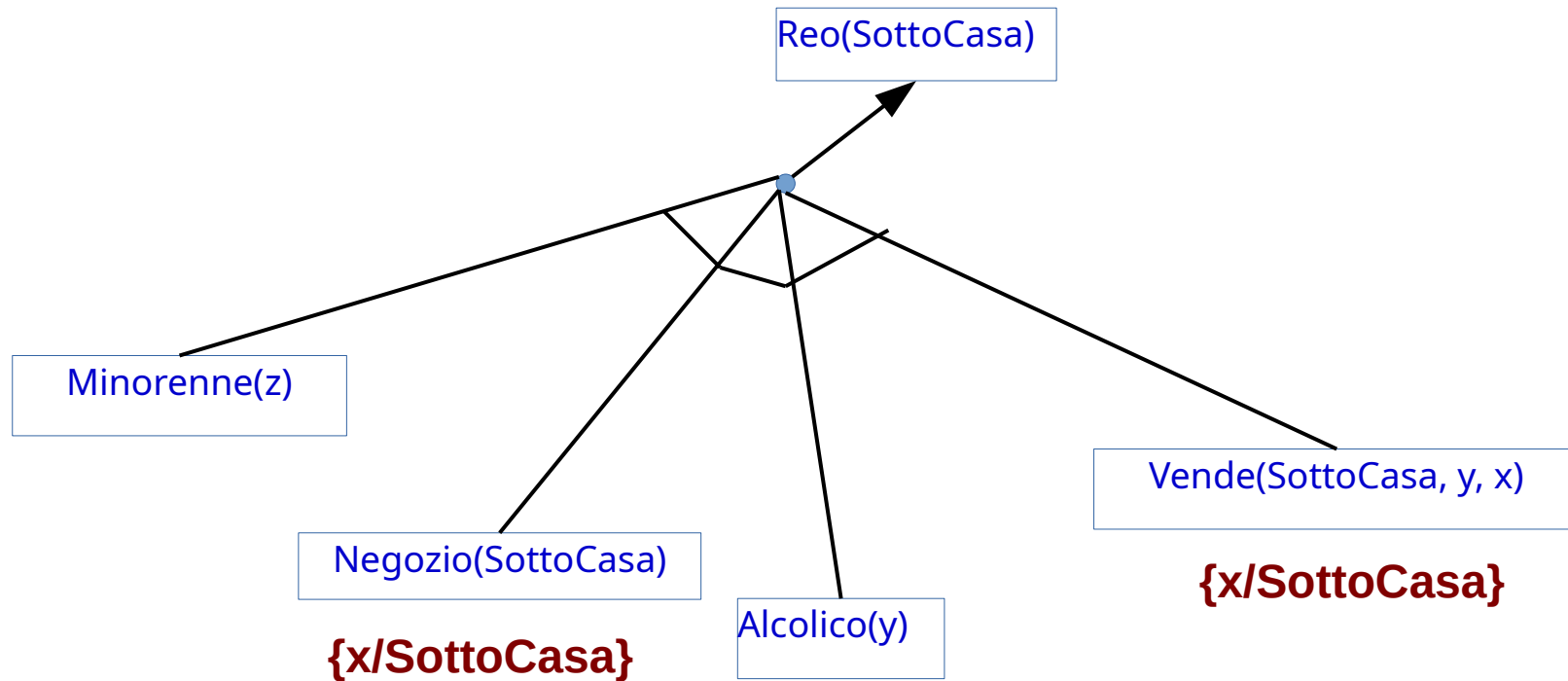
Si applica la sostituzione al corpo di C1, tutti i congiunti diventano obiettivi da dimostrare e vengono inseriti nello stack:

Stack: $\text{Negozio}(\text{SottoCasa}), \text{Vende}(\text{SottoCasa}, y, z), \text{Alcolico}(y), \text{Minorenne}(z)$

Esempio

C1) $\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$

Stack: $\text{Negozio}(\text{SottoCasa}), \text{Vende}(\text{SottoCasa}, y, z), \text{Alcolico}(y), \text{Minorenne}(z)$

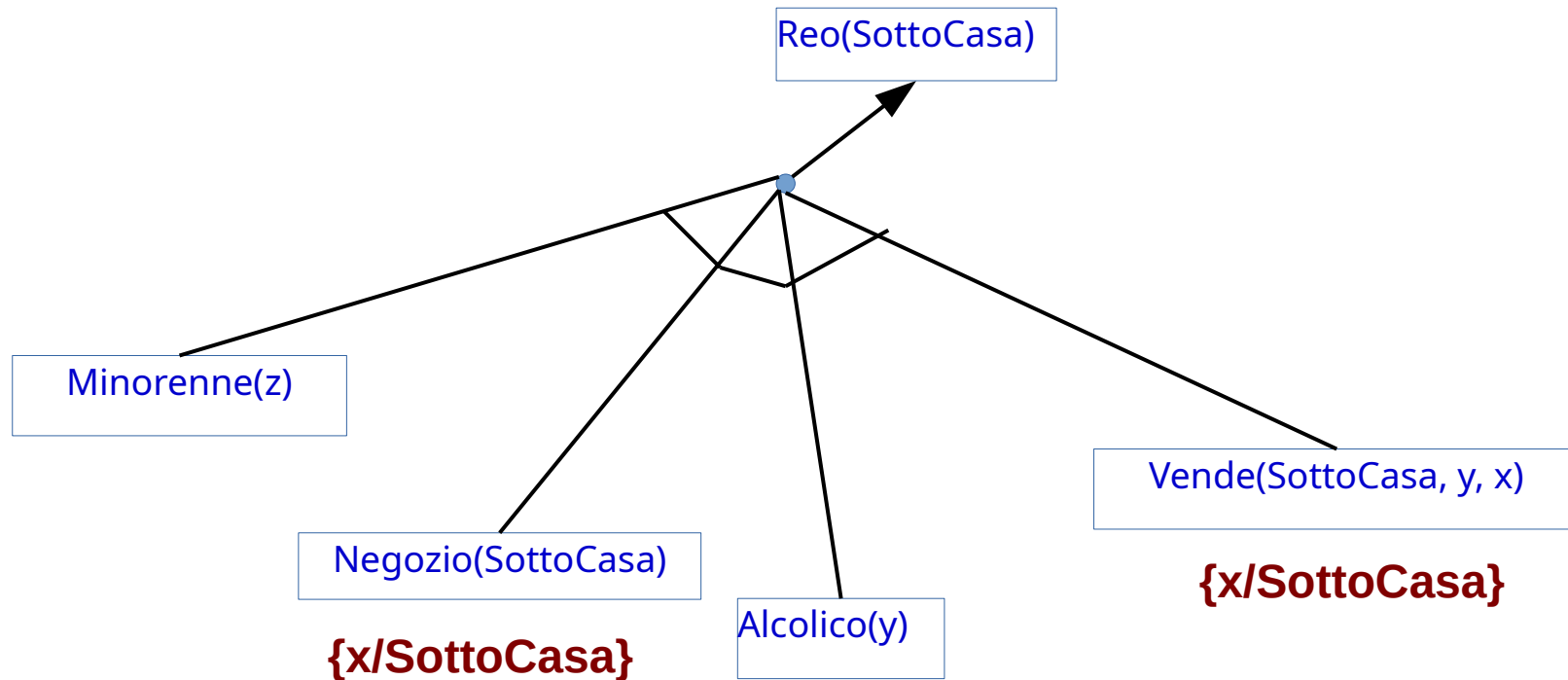


Esempio

C1) $\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$

Stack: $\text{Vende}(\text{SottoCasa}, y, z), \text{Alcolico}(y), \text{Minorenne}(z)$

Risolviamo ricorsivamente $\text{Negozio}(\text{SottoCasa})$

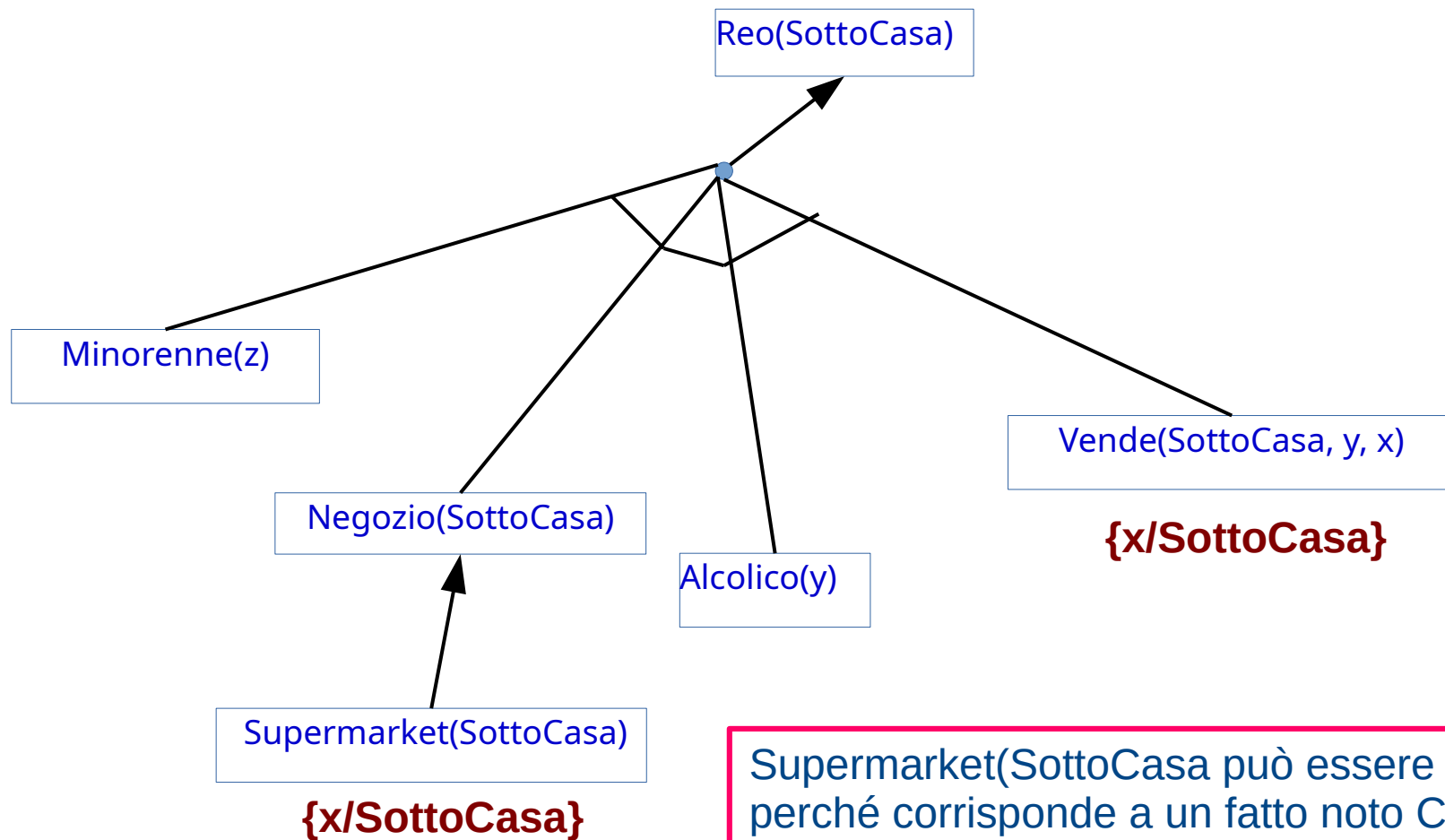


unifica con la testa della clausola C7) $\text{Supermarket}(x) \Rightarrow \text{Negozio}(x)$

Esempio

C1) $\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$

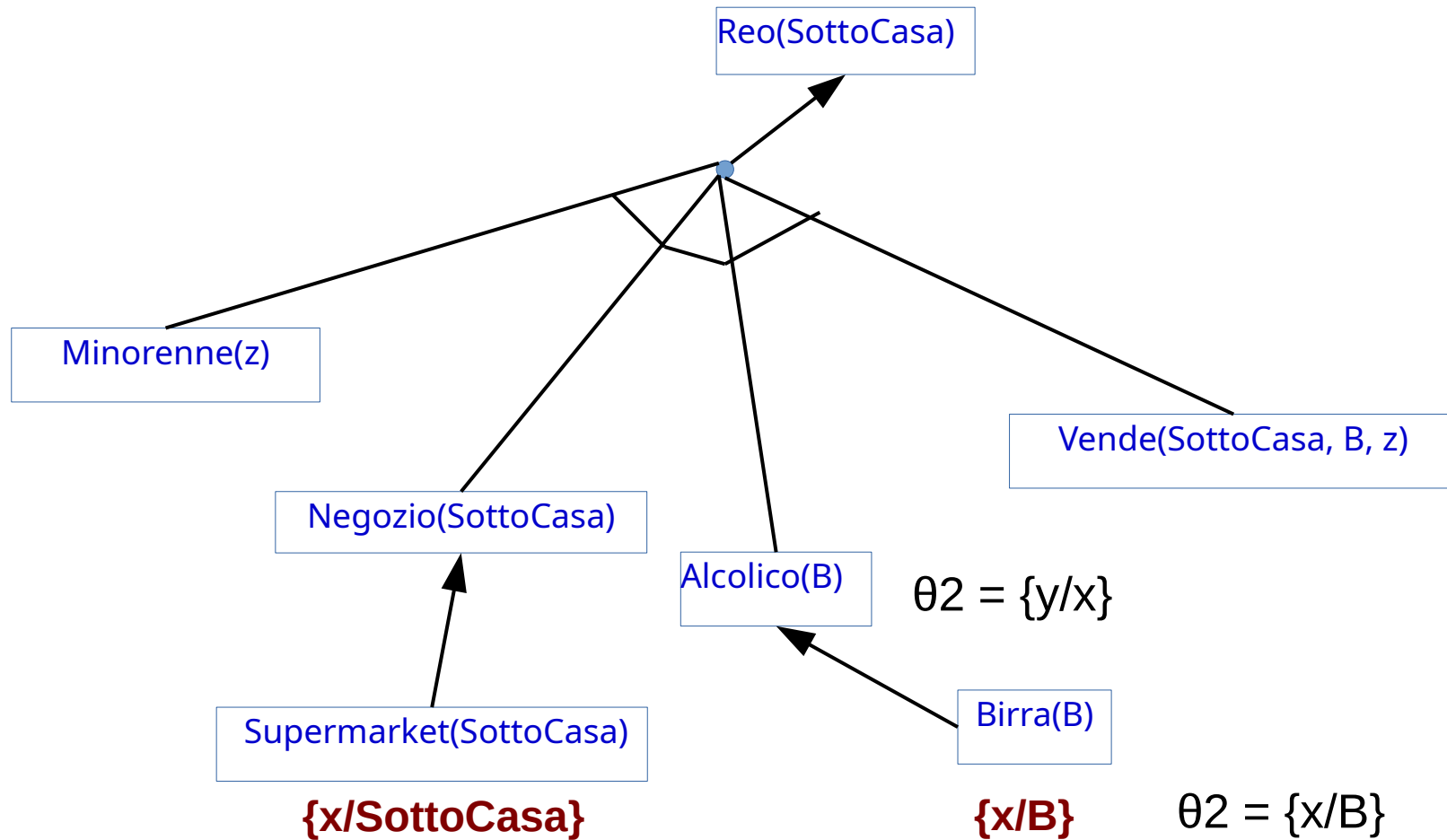
Stack: ~~Supermarket(SottoCasa)~~, ~~Vende(SottoCasa, y, z)~~, ~~Alcolico(y)~~, ~~Minorenne(z)~~



Esempio

C3) Birra(B)

C5) Birra(x) $\Rightarrow$ Alcolico(x)



Composizione delle sostituzioni

- l'algoritmo BC applica la composizione delle sostituzioni:

$$\text{COMPOSE}(\theta_1, \theta_2) = \theta_3$$

- Vale la proprietà:

$$\text{SUBST}(\text{COMPOSE}(\theta_1, \theta_2), F) = \text{SUBST}(\theta_2, \text{SUBST}(\theta_1, F))$$

- Nell'esempio abbiamo:

$\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$

$\text{Birra}(x) \Rightarrow \text{Alcolico}(x)$

$\text{Birra}(B)$

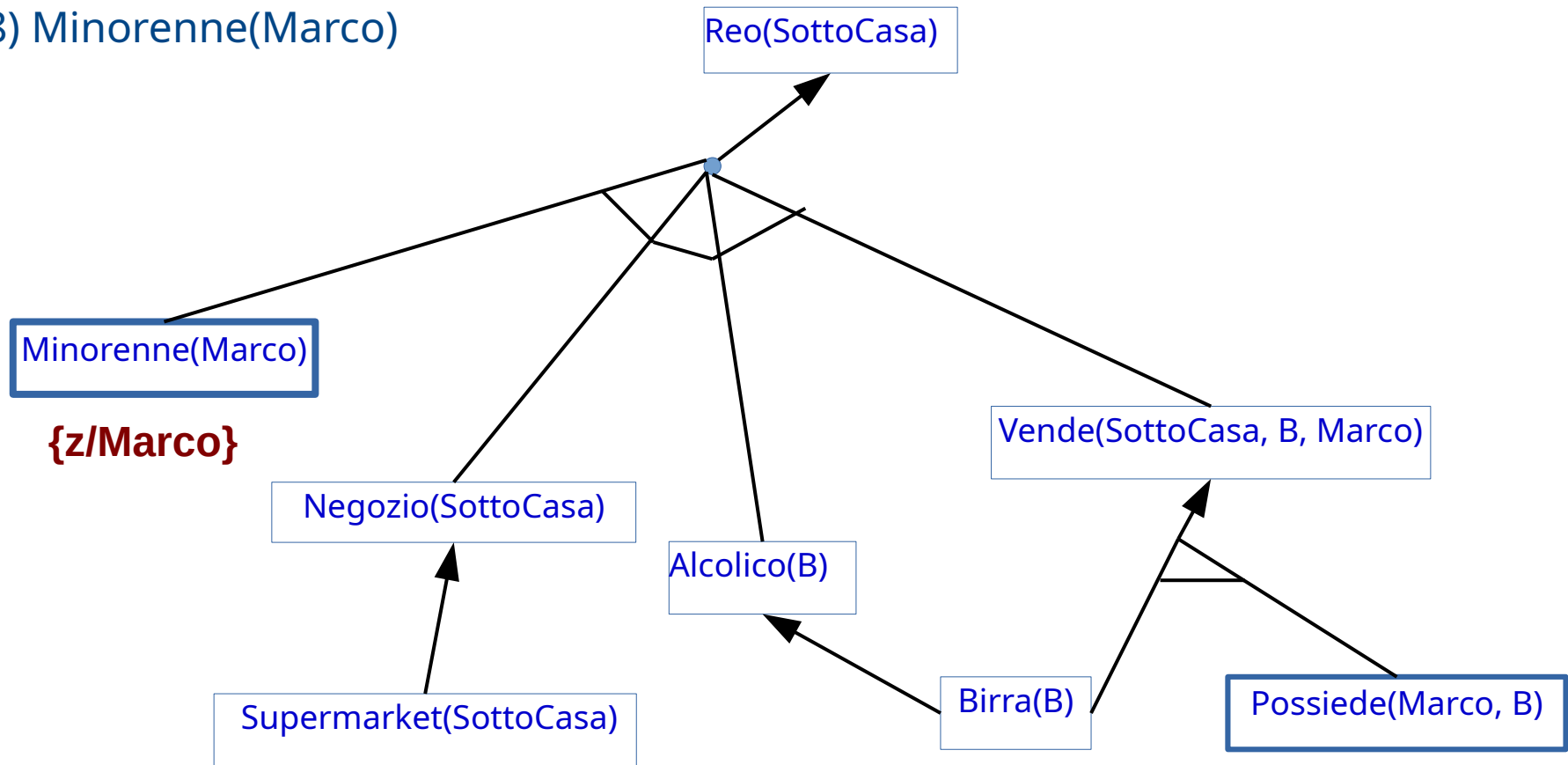
- Occorre concatenare le sostituzioni per effettuare l'inferenza corretta

Esempio

C2) Possiede(Marco, B)

C4) Possiede(Marco, x) $\wedge$ Birra(x) $\Rightarrow$ Vende(SottoCasa, x, Marco)

C8) Minorenne(Marco)



Valutazione del backward chaining

- **Corretto**
- **Incompleto** in quanto implementa una *strategia depth-first*, può incorrere in loop infiniti e generare stati ripetuti
- Come la versione per il calcolo proposizionale è più efficiente del forward chaining